

Отчёт по анализу APK: Amazon Relay

Разработчик: Amazon Mobile LLC. Package: com.amazon.relay. Версия 2.44.233 (versionCode 2442330). Разбор выполнен заново и независимо от предыдущего отчёта: ХАРК скачан с APKPure, базовый APK имеет SHA-256 7c016e60e1926f27e7137b8956db3ad961a1222a384bb31579ce38b7b305b1bd (совпадает с prepare.json), все 15 файлов dex распакованы заново через jadx 1.5.3 — это 7 743 файла в пакете com.amazon.relay. APK подписан сертификатом Amazon Services LLC (CN = Amazon Services LLC, O = Amazon Services LLC, L = Las Vegas, ST = Nevada, отпечаток SHA-256 2F:19:AD:EB:28:4E:B3:6F:7F:07:78:61:52:B9:A1:D1:4B:21:65:32:03:AD:0B:04:EB:BF:9C:73:AB:6D:76:25). Дополнительно прочитаны AndroidManifest.xml, strings.xml, все файлы версий из META-INF, папка assets и строки всех dex.

Поимённый разбор срабатываний локального скана

Каждое место, на которое сработал предварительный поиск, вскрыто отдельно: откуда берётся адрес, кто вызывает этот код и что получает пользователь.

com/amazon/relay/carrier/InternalWebViewKt.java — loadUrl

В файле есть функция InternalWebView(url), которая создаёт окно браузера внутри экрана, включает JavaScript, ставит обычный WebViewClient без переопределений и вызывает loadUrl(url). Адрес приходит только снаружи, в параметре. По всему APK у этой функции ровно один вызов — com/amazon/relay/stc/views/list/ComposableSingletons\$HomeScreenKt.java, где адрес читается из навигационного аргумента «weburl» маршрута Screen.InternalWebViewLinks («...://internalWebViewLinks/{weburl}»). Переход на этот маршрут в приложении тоже один: экран подтверждения краткосрочного контракта (HomeScreenKt\$ShortTermContracts\$3\$1\$3) берёт ссылку из текста соглашения, кодирует её через URLEncoder и передаёт в маршрут. Сам текст соглашения собирается в ContractConfirmationKt, и там ссылка вписана в код целиком: https://relay.amazon.com/profile/agreements?ref=owp_nav_co_doc_agr&tabId=programPolicies&documentId=relay_us_program_policies_april_2022 — это правила программы для перевозчика. Константа INITIAL_URL в этом же файле равна <https://relay-preprod-iad.iad.proxy.amazon.com> и используется только отладочным просмотрщиком адресов (метрики MANUAL_DEBUG_WEB_PAGE_LOAD_*), куда адрес вводится руками. Вспомогательная функция validateUrlPrefix лишь дописывает «https://», если схема не указана. Ни одна строка здесь не приходит с сервера.

com/amazon/relay/carrier/WebViewFragment.java — loadUrl

Это общий фрагмент портала Relay. В onCreateView адрес берётся из аргументов фрагмента (arguments.getParcelable(ARG_URI)), к нему дописывается параметр navless=true, в User-Agent окна добавляется префикс «RelayMobile-Android-<версия приложения>», включаются JavaScript, DOM-хранилище, база и доступ к файлам, после чего вызывается loadUrl. Обработчиком страниц назначается MAPAndroidWebViewClient — это клиент библиотеки входа Amazon (MAP), которому через MAPAndroidNewWebViewHelper передаётся параметр pagelId со значениями amzn_relay_desktop_us, _ca, _mx, _eu, _jp, _uk в зависимости от страны аккаунта; он нужен, чтобы окно подхватило уже выполненный вход в Amazon. Фрагмент создаётся из восемнадцати экранов приложения (загрузочная доска, рейсы, контракты, водители, выплаты, документы компании, профиль, аукционы, награды, уведомления, поддержка, производительность, активы, пользователи площадки, «Разместить машину», создание аккаунта перевозчика, помощник Relay, учебный центр). Во всех случаях адрес строит класс CarrierUrlUtil из вшитых хостов: relay.amazon.com для Северной Америки, relay.amazon.co.uk и relay.amazon.in для Европы и Индии, relay-preprod-iad.iad.proxy.amazon.com и relay-preprod-uk-dub-dub.dub.proxy.amazon.com для внутренних предпродакшен-сборок. Путь дописывается в коде (tours, loadboard, contracts, drivers, payments, profile/carrier, profile/agreements, cases/sc, learning и так далее). Сервер не может подставить сюда свой адрес: хост выбирается перечислением региона, а не ответом по сети.

com/amazon/relay/config/LaunchUrlNavigationItem.java — CustomTabsIntent

Класс умеет открыть внешнюю вкладку Chrome: он собирает CustomTabsIntent, красит панель в цвета приложения, включает скрывание адресной строки и подставляет в intent адрес из поля

launchUrl, которое задаётся в конструкторе. Поиск по всему распакованному APK даёт ноль обращений к этому классу: его никто не создаёт и ни в один список пунктов меню он не попадает. То есть это оставшийся в сборке неиспользуемый код, который при работе приложения не выполняется, и значение launchUrl в APK вообще ниоткуда не берётся.

com/amazon/relay/pod/GenericDocumentUploadManager.java — loadUrl

Здесь срабатывание поиска оказалось совпадением по буквам: слова loadUrl в файле нет, есть uploadUrl и getUploadUrl, внутри которых «loadUrl» является частью слова. Окна браузера в классе нет совсем — он отвечает за отправку фотографий и сканов документов о доставке. Логика такая: вызывается gatewayDocumentsClient.requestUploadDocument() к шлюзу Amazon, в ответе приходят два поля — documentKey и uploadUrl, — после чего FileUrlOperations.upload кладёт файл или картинку по этому адресу обычным HTTP-запросом. Это стандартная предподписанная ссылка на хранилище Amazon S3 со сроком жизни; пользователю она не показывается и ни в каком браузере не открывается.

Остальные подсказки из cloaking_hints.md

GatePassFragment и FragmentGatePassBinding — это пропуск на ворота склада: фрагмент получает из GatePassRepo номер груза, день и время прибытия и рисует QR-код через Bindings.generateQRCode, отправляя метрику RLM_QR_CODE_LAUNCHED_FROM_YMS_CHECKIN. Слово «gate» здесь означает физические ворота склада, а не отбор трафика. AppInitializationDelegate перебирает зарегистрированные обработчики инициализации и вызывает у каждого onAppInitialize, ошибки просто пишет в журнал; никакого HTTP-запроса за решением «что показать» он не делает. ConfigManager.startLaunchFlow — это конечный автомат запуска: AUTH, затем DRIVER_ONBOARDING, затем DRIVER_INVITE и в конце HomeActivity. PendoGateActivity — экран из библиотеки подсказок Pendo (sdk.pendo.io), через который она перехватывает жесты для показа своих обучающих всплывашек. AgeBlockedActivity показывает заглушку, когда Google Play сообщил, что аккаунт принадлежит несовершеннолетнему: сигнал берётся у AgeSignalsManager из Google Play (com.google.android.play.agesignals), а кнопки ведут либо в системные настройки, либо обратно на AppLaunchActivity.

SDK / стек

Параметр	Значение
Название приложения	Amazon Relay (в Google Play — «Amazon Relay»; application-label в APK — «Amazon Relay», строка app_name в strings.xml; package com.amazon.relay; versionName 2.44.233, versionCode 2442330)
Android Gradle Plugin	8.12.2 (META-INF/com/android/build/gradle/app-metadata.properties)
minSdk	29
targetSdk	36
Kotlin	да, 2.0.21
Web View	да
Custom Tabs	да
Рекламные сети	нет
Аналитика	Pendo (продуктовые подсказки и события), Amazon Kinesis (KinesisMetricsRecorder, события запуска и сбоев), Amazon Minerva, Amazon DCP Client Metrics, Amazon Weblab (A/B-варианты), AWS Pinpoint через Amplify Analytics, Amazon Device CrashManager, Firebase Cloud Messaging (push-уведомления)

Параметр	Значение
Permissions	amazon.permission.COLLECT_METRICS, android.permission.ACCESS_BACKGROUND_LOCATION, android.permission.ACCESS_COARSE_LOCATION, android.permission.ACCESS_FINE_LOCATION, android.permission.ACCESS_NETWORK_STATE, android.permission.ACCESS_NOTIFICATION_POLICY, android.permission.ACCESS_WIFI_STATE, android.permission.AUTHENTICATE_ACCOUNTS, android.permission.CAMERA, android.permission.CREDENTIAL_MANAGER_QUERY_CANDIDATE_CREDENTIALS, android.permission.FOREGROUND_SERVICE, android.permission.FOREGROUND_SERVICE_LOCATION, android.permission.GET_ACCOUNTS, android.permission.INTERNET, android.permission.MANAGE_ACCOUNTS, android.permission.POST_NOTIFICATIONS, android.permission.READ_EXTERNAL_STORAGE, android.permission.READ_PHONE_STATE, android.permission.RECEIVE_BOOT_COMPLETED, android.permission.USE_BIOMETRIC, android.permission.USE_CREDENTIALS, android.permission.USE_FINGERPRINT, android.permission.VIBRATE, android.permission.WAKE_LOCK, android.permission.WRITE_EXTERNAL_STORAGE, com.amazon.appmanager.preload.permission.READ_PRELOAD_DEVICE_INFO_PROVIDER, com.amazon.dcp.config.permission.DYN_CONFIG_VALUES_UPDATED, com.amazon.dcp.metrics.permission.METRICS_PERMISSION, com.amazon.dcp.sso.permission.account.changed, com.amazon.dcp.sso.permission.AmazonAccountPropertyService.property.changed, com.amazon.dcp.sso.permission.CUSTOMER_ATTRIBUTE_SERVICE, com.amazon.dcp.sso.permission.MANAGE_COR_PFM, com.amazon.dcp.sso.permission.USE_DEVICE_CREDENTIALS, com.amazon.identity.auth.device.perm.AUTH_SDK, com.amazon.identity.permission.CALL_AMAZON_DEVICE_INFORMATION_PROVIDER, com.amazon.identity.permission.CAN_CALL_MAP_INFORMATION_PROVIDER, com.amazon.identity.permission.GENERIC_IPC, com.amazon.relay.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION, com.google.android.c2dm.permission.RECEIVE

Параметр	Значение
Libraries	Jetpack Compose (runtime, runtime-livedata, runtime-saveable, ui, ui-text, ui-graphics, ui-unit, ui-tooling, foundation, foundation-layout, material, material3, material-icons-core, material-icons-extended, navigation-compose, constraintlayout-compose, activity-compose, lifecycle-runtime-compose, lifecycle-viewmodel-compose), androidx (appcompat, core, core-ktx, activity, activity-ktx, fragment, fragment-ktx, annotation-experimental, autofill, biometric, browser, camera-core, camera-camera2, camera-lifecycle, camera-video, camera-view, camera-featurecombinationquery, cardview, constraintlayout, coordinatorlayout, credentials, credentials-play-services-auth, cursoradapter, customview, customview-poolingcontainer, databinding (library, baseAdapters, databindingKtx, viewbinding), drawerlayout, dynamicanimation, emoji2, exifinterface, graphics-path, interpolator, legacy-support-v4, lifecycle (extensions, livedata, livedata-ktx, livedata-core, process, reactivestreams, runtime, runtime-ktx, service, viewmodel, viewmodel-ktx), loader, media, media3, navigation (common, runtime, runtime-ktx, fragment, fragment-ktx, ui, ui-ktx), percentlayout, preference, preference-ktx, profileinstaller, recyclerview, recyclerview-selection, room-runtime, room-ktx, room-rxjava2, savedstate, security-crypto, sqlite, sqlite-framework, startup-runtime, tracing, transition, vectordrawable, versionedparcelable, viewpager, viewpager2, window, work-runtime, work-runtime-ktx, work-rxjava2), Dagger 2 (dagger, dagger-android, dagger-android-support), kotlin.coroutines 1.6.4 (core, android, rx2, reactive, guava, play-services), kotlinx.serialization, RxJava 2 и RxAndroid, RxBinding (JakeWharton), Retrofit 2 с адаптерами JakeWharton, OkHttp 3 и Okio, Gson, Jackson (fasterxml), Apollo GraphQL, AWS SDK for Android (com.amazonaws: Cognito, Pinpoint, Kinesis, Arcus, S3-клиент), AWS Amplify (com.amplifyframework: analytics, api, auth, storage, geo, notifications, datastore), AWS SDK Kotlin и Smithy (aws.sdk, aws.smithy, software.amazon), библиотеки Amazon (identity/MAP, dcp, device.crashmanager, client.metrics, minerva, weblab, meridian — набор UI-компонентов, ion, optimus, relayloadmanagementviewsservice), Pendo (sdk.pendo.io), Firebase (messaging, installations, iid, components, datatransport, encoders, heartbeatinfo, tracing), Google Play Services (auth, base, common, cloudmessaging, location, fido, identitycredentials, phenotype, security, signin, stats, tasks, vision), Google Play age-signals и Play Core, ML Kit (common, vision) и barhopper, ZXing, Google Tink, Accompanist, Flexbox, Material Components, HERE SDK (com.here: sdk, odnp, posclient, services, time), Coil, TensorFlow Lite, Time4J (net.time4j), Apache Commons, commonmark, SLF4J, libphonenumber, ReLinker (getkeepsafe), ProcessPhoenix (JakeWharton), NYTimes Store, Snapper (dev.chrisbanes), ByteBuddy, JUnit, Hamcrest и Lombok (остатки сборочных зависимостей)
Подозрительные домены	нет
SharedPreferences	да. Приложение использует несколько именованных хранилищ: RATING_SHARED_PREF (состояние просьбы оценить приложение), location_sender_prefs с ключом location_sender_bucket_time (сколько секунд копить точки маршрута перед отправкой), ntp_prefs с ключами ntp_offset, last_refresh_timestamp и last_clock_skew (поправка часов устройства), device_info_reporter с ключом last_reported_ms (когда последний раз отправляли сведения о модели устройства), daily_notification (разовое напоминание за сутки), com.amazon.relay_preferences с ключом NAV_VISUAL_NAVIGATOR_DEBUG_MODE_KEY (отладка навигатора), а также отдельное хранилище на каждый модуль: PersistedStorageFactory создаёт SharedPreferences с именем модуля и SharedPreferencesService при чтении переносит старые значения из общего хранилища в модульное. Там лежат признаки показанных экранов, выбранная страна и среда, состояние запросов разрешений и значения флагов функций
Есть ли клоака	нет
Подозрительные слова	gateway (924 совпадения — это собственные шлюзы Amazon: gatewayUrl, relayGatewayNetworkClient, gatewayDocumentsClient, InspectionClient), gate (9 совпадений — пропуск на ворота склада GatePass и строка «gate» в наблюдателе условий слежения за маршрутом), redirect (7 совпадений — шаблон серверной вёрстки RedirectView/CommonTemplateName.redirect и обработка переадресации расписания), proxy (21 совпадение — внутренние предпродакшен-хосты вида relay-preprod-iad.iad.proxy.amazon.com). Слов cloak, cloaker, whitepage, white_page, offerwall, is_bot, bot, whitelist, blacklist, landing, preland, keitaro, binom, smartlink, postback, clickid в коде, строках и файлах assets нет

Точка входа: что происходит при запуске

В манифесте классом приложения указан `com.amazon.relay.RelayApp`. Его `attachBaseContext` делает ровно одно — `MultiDex.install(this)`, потому что код не помещается в один файл dex. В `onCreate` по порядку: запускается библиотека подсказок Pendo (`PendoManager.setup` с вшитым рабочим ключом), собирается граф зависимостей Dagger и внедряется в приложение, вызывается `initialize()` у модуля Foundation, инициализируется логика входа `authLogic.init()`, к жизненному циклу подключается `AppLifecycleObserver`, в фоне стартуют три задачи (инициализация Weblab, старт сессии Pendo для уже вошедшего пользователя и `AppInitializationDelegate.onAppInitialize`), затем `configManager.runAppInitializers()` и запись события запуска в метрики Amazon Kinesis. Никакого обращения на внешний адрес «за решением, что показать» в этой цепочке нет.

Стартовая Activity — `com.amazon.relay.startup.AppLaunchActivity`: у неё и `MAIN/LAUNCHER`, и проверенные ссылки `https` на `relay.amazon.com` и `relay.amazon.co.uk` с `autoVerify`. `AppLaunchViewModel` решает, какой следующий шаг нужен: вход в тестовом режиме, выбор среды, выбор страны, приветственный экран или «готово». После «готово» вызывается `ConfigManager.startLaunchFlow`, который ведёт пользователя по цепочке `AUTH` → `DRIVER_ONBOARDING` → `DRIVER_INVITE` и открывает `HomeActivity`. Вторая точка входа — `DeepLinkActivity` со схемой `amazonrelay`. Адрес вида `amazonrelay://setup/<строка в Base64>` расшифровывается в `DeepLinkResolver` через `Base64.getDecoder` и разбирается Gson в объект с полями страна, среда, логин и пароль — это служебный вход для автотестов, остальные пути этой схемы просто переводят на нужную вкладку приложения.

Сетевой слой: адреса и поля запроса

Весь собственный обмен идёт на шлюзы Amazon, вшитые в модуль Foundation: `https://api.relay.amazon.com/` для Северной Америки, `https://api.relay.amazon.eu/` для Европы, `https://api.relay.amazon.jp/` для Японии и зеркала `https://cloudfront.na.api.relay.amazon.dev/`, `cloudfront.eu.api.relay.amazon.dev`, `cloudfront.fe.api.relay.amazon.dev`. Клиенты описаны через Retrofit, тело — JSON (Gson или `kotlinx.serialization`).

К каждому запросу `AuthenticationInterceptor` добавляет пять полей. `User-Agent` собирается по шаблону «пакет/версия (`os.name`; U; Android <версия ОС>; <язык>-<страна>; <модель> Build/<идентификатор сборки>; <бренд>) <ширина>X<высота> <производитель> <модель>». `x-amz-access-token` и `x-relay-access-token` содержат один и тот же токен входа Amazon. `x-amz-request-id` — новый случайный идентификатор на каждый запрос. `x-relay-installation-id` — постоянный идентификатор установки приложения. Если токена нет, перехватчик не отправляет запрос вообще, а возвращает коду приложения искусственный ответ 403.

Полный список адресов, встречающихся в собственном коде приложения: `relay.amazon.com` (портал, картинки инструкций, страницы соглашений, уведомления), `relay.amazon.co.uk` и `relay.amazon.in` (европейский и индийский порталы), `relay-preprod-iad.iad.proxy.amazon.com` и `relay-preprod-uk-dub-dub.dub.proxy.amazon.com` (внутренние предпродакшен-стенды), `api.relay.amazon.com`, `api.relay.amazon.eu`, `api.relay.amazon.jp` и три зеркала `*.api.relay.amazon.dev`, `na.gamma.api.relay.amazon.dev` (тестовый пример в коде), `rss.app-resources.relay.amazon.com` и три раздачи CloudFront для модуля поддержки, одна раздача CloudFront для веб-страницы чата переговоров, `dfz3xbn3chdu5.cloudfront.net` (картинка значка), `pki.amazon.com` (сертификаты Amazon), `www.amazon.com/privacy/data-deletion` и `www.amazon.com/gp/help/customer/display.html` (справка), `byod-amm.router.hereapi.com` (расчёт маршрута в навигаторе HERE), `play.google.com/store/apps/details` (переход в магазин для обновления) и `atsescalation.com`. Последний — линия конфиденциальных обращений Amazon Partner Escalation: на экране «Сообщить о проблеме на рабочем месте» текст строки `optimus_r4d_pr_report_wrongdoing_body` заканчивается видимой ссылкой `atsescalation.com`, и только по нажатию на неё открывается системный браузер. Никаких данных устройства при этом не передаётся. В файлах `domain_checks.json` и `domain_checks.md` проверок доменов для этого приложения нет, поэтому в таблице «Подозрительные домены» стоит «нет».

Встроенное окно сайта и внешний браузер: полный перечень

Окон сайта внутри приложения пять, и адрес каждого известен заранее. Первое — общий `WebViewFragment` портала `Relay`, разобранный выше. Второе — `InternalWebView` с правилами программы, тоже разобранный выше. Третье — учебный центр: `LearningCenterNavActivity` открывает `relay.amazon.com/learning`, а его `shouldOverrideUrlLoading` сверяет хост нажатой ссылки со списком `LEARNING_CENTER_EXTERNAL_DOMAINS` из настроек `Amazon` и, если хост в списке, отдаёт ссылку системному браузеру через `Intent ACTION_VIEW`. Это список разрешённых сайтов для исходящих ссылок учебных материалов, а не отбор пользователей. Четвёртое — модуль поддержки `Smart Support`: он грузит `https://rss.app-resources.relay.amazon.com`, включает `JavaScript` и регистрирует свои JS-мосты; региональные сборки интерфейса лежат на `d195g4mjt1ak18.cloudfront.net`, `drc5xd6v6br06.cloudfront.net` и `d2xlfrqnwy716f.cloudfront.net` — идентификаторы раздач вшиты в `ISmartSupportDaggerEnvironmentModule`. Пятое — чат переговоров по ставке на загрузочной доске: `ChatbotConfig` хранит идентификатор раздачи `CloudFront` для каждой пары «регион + среда» и собирает адрес как «https://» + идентификатор + «.cloudfront.net/negotiation-chat.html».

Внешняя вкладка браузера (`Custom Tabs`) реально открывается в одном месте — `HttpRouter.launchBrowserActivity`. Класс `LaunchUrlNavigationItem`, который тоже умеет это делать, в приложении не используется. Кроме того, системный браузер через `Intent ACTION_VIEW` открывают учебный центр, ссылка `atsescalation.com` и переход в `Google Play` за обновлением.

Серверная вёрстка экранов и шаблон «redirect»

В приложении есть механизм, при котором сервер `Amazon` присылает описание экрана в `JSON`, а приложение его рисует (пакет `com.amazon.relay.views`). Запрос делает `ViewCollectionService` через `relayGatewayNetworkClient`, то есть на тот же шлюз `api.relay.amazon.*` и с теми же пятью полями авторизации. Среди шаблонов есть и «redirect»: `RedirectView` при появлении на экране сразу выполняет именованную ссылку из описания (поля `linkName`, `callbackLinkName`, `onFailureLinkName`). Имя ссылки превращается в адрес через `NamedLinkHandler` и уходит в цепочку маршрутизаторов `RouterChain`; `HttpRouter`, если такого экрана внутри приложения нет и к ссылке не приложены данные запроса, открывает адрес во внешней вкладке браузера, а при ответе с кодом переадресации берёт заголовок `location` и прогоняет новый адрес по той же цепочке.

Это единственное место, где адрес перехода приходит с сервера, поэтому оно разобрано отдельно. Отправителем здесь выступает авторизованный шлюз `Amazon`, к которому приложение обращается уже с токеном водителя или перевозчика; без входа запрос вообще не уходит. Присланные экраны — это рабочие карточки рейса, задания на погрузку и выгрузку, формы осмотра и подтверждения, и к ним не прилагается никакая вторая версия приложения. В ответе нет ни поля с рекламной страницей, ни признака «показать обычное приложение». Приложение не отправляет перед этим ни рекламный идентификатор, ни данные о том, эмулятор это или нет; решение о содержимом принимается по назначенному водителю рейсу, а не по свойствам устройства.

Удалённые настройки

Удалённый конфиг есть, но он выдаёт только флаги. `ICConfigurationNetworkClient` запрашивает «foundation/config» на том же шлюзе `Amazon`, ответ `GetConfigurationResponse` состоит из пространств имён и наборов значений: `booleanValues`, `stringValues`, `numberValues`, `stringSetValues` и `serializedObjectValues`. `ConfigurationCache` обновляет их не чаще раза в десять секунд. Для постепенной раскатки используется `Configuration.evaluateRollout`: от идентификатора считается `MD5` и берётся остаток, чтобы включить функцию у заданного процента установок. Через эти флаги, например, включается суточный отчёт о модели устройства (`REPORT_DEVICE_METRIC_PAYLOAD`) и задаётся список внешних доменов учебного центра. Поля со ссылкой, по которой надо открыть страницу вместо приложения, в ответе нет. Вторая система — `Weblab`, собственные A/B-тесты `Amazon`: по имени эксперимента возвращается буква варианта, клиент представляется набором «RelayAndroid», версия, `Android`, версия ОС, производитель и модель; единственный встреченный в коде эксперимент называется `RELAY_PRE_TRIP_SAFETY_VIDEO_1463916` и отвечает за показ ролика по безопасности перед рейсом.

Строки, Base64 и сборка адресов

Расшифровка Base64 применяется в одном месте — в DeepLinkResolver для служебной ссылки amazonrelay://setup, где в строке закодирован JSON с настройками тестового входа. Все остальные адреса собираются открытым текстом: CarrierUrlUtil складывает схему, вшитый хост и путь через Uri.Builder, ChatbotConfig и модуль Smart Support подставляют вшитые идентификаторы раздач CloudFront, модуль Foundation хранит базовые адреса шлюзов строками. Ключи Pendo — это два JWT (рабочий и отладочный), внутри которых лежит ключ приложения Pendo. Склейки адреса из кусков, спрятанных в ресурсах или в файлах assets, нет; в assets лежат только карты, справочники городов, модели распознавания, иконки и голосовые файлы, HTML- и JS-файлов там нет.

Проверки устройства, которые есть в приложении

Два места действительно смотрят на устройство, и оба разобраны. StartupValidations при запуске сверяет имя пакета, отпечаток SHA-256 сертификата подписи и ключи metadata из манифеста со списками, которые приходят в настройках Amazon; если проверка не прошла и включён флаг BLOCK_APP_LAUNCH_ON_STARTUP_VALIDATION_FAILURE, приложение просто перестаёт работать. Это защита от пересборки и подмены APK, а не выбор содержимого: альтернативного экрана в этой ветке нет. FraudSignalManagerImpl считает подозрительными поддельное местоположение, эмулятор и root-доступ, и его единственный потребитель — PrimaryDeviceSwitchManagerImpl, который по этому признаку отказывается переносить роль основного устройства водителя по координатам. Это борьба с подделкой GPS при подтверждении рейса. Чтения рекламного идентификатора устройства в приложении нет совсем: класса AdvertisingIdClient в APK не существует.

Отдельно стоит упомянуть DeviceInfoReporter: раз в сутки, если включён флаг REPORT_DEVICE_METRIC_PAYLOAD, он собирает производителя, модель, версию Android, уровень API, версию приложения и языковую метку и отправляет их одной метрикой ANDROID_DEVICE_INFO_PAYLOAD в систему метрик Amazon. Это телеметрия парка устройств: она уходит после запуска, а не перед показом содержимого, и ответа, от которого зависел бы экран, на неё не приходит.

Название в магазине и внутри APK

В Google Play приложение называется «Amazon Relay». В APK атрибут android:label у приложения ссылается на строку app_name, а в strings.xml app_name равен «Amazon Relay». Названия совпадают полностью. Пакет com.amazon.relay, подпись — сертификат Amazon Services LLC. Подмены вывески, когда в магазине одно имя, а на устройстве другое, здесь нет.

Реклама

Рекламных сетей в приложении нет: ни AdMob и Google Mobile Ads, ни AppLovin, ни Unity Ads, ни ironSource, ни Facebook Audience Network, ни AppsFlyer или Adjust. Пакета com.google.android.gms.ads в APK нет. Из Firebase подключены только доставка push-уведомлений и вспомогательные части (installations, iid, datatransport); модуль Firebase Analytics представлен лишь интерфейсом-заглушкой AnalyticsConnector без реализации.

Вывод

Это официальное приложение Amazon Relay для водителей и перевозчиков, подписанное сертификатом Amazon Services LLC. После повторного полного разбора точки входа, всего сетевого слоя, всех пяти окон сайта, обеих точек запуска внешнего браузера, механизма серверной вёрстки экранов и удалённых настроек развилки «одним — рекламная страница, другим — обычное приложение» в коде нет: тихого запроса на незнакомый сервер перед показом содержимого нет, ответа со ссылкой, от которого зависел бы экран, нет, отбора по боту, стране или языку ради показа оффера нет, рекламных сетей и чтения рекламного идентификатора нет, а имя в магазине совпадает с именем внутри APK. Все четыре срабатывания предварительного поиска объяснены по

месту: два окна сайта получают вшитые адреса портала Amazon, класс с внешней вкладкой браузера в приложении не используется, а четвёртое срабатывание оказалось совпадением по буквам внутри слова `uploadUrl`. Поэтому в таблице «Есть ли клоака» стоит «нет».